PROJECT REPORT

TASK 20: SORT DATASET USING PANDAS

1. Abstract

This project demonstrates dataset sorting using Python and Pandas. A sample student performance dataset is used to sort numerical and categorical columns in ascending and descending order. The project also demonstrates sorting by multiple columns and explains how sorting helps organize and analyze data.

2. Introduction

Sorting is an important operation in data analysis because it organizes records according to a chosen column or set of columns. Numerical sorting orders values from low to high or high to low, while categorical sorting organizes text values alphabetically.

Pandas provides the `sort_values()` function to sort records according to one or more columns. This project demonstrates these capabilities using a student performance dataset in Google Colab.

3. Objectives

The objectives are:

- To perform ascending numerical sorting.
- To perform descending numerical sorting.
- To sort categorical values alphabetically.
- To demonstrate sorting using multiple columns.
- To understand the role of sorting in data analysis.

4. Tools and Technologies

Python: Used to execute the sorting operations.

Pandas: Used to create and sort the DataFrame.

NumPy: Available for numerical data handling.

Google Colab: Used as the notebook execution environment.

CSV: Used to store and export the sample dataset.

5. Dataset Description

The project uses 12 manually created fictional student records.

The dataset contains six columns: student ID, name, department, age, marks, and

attendance.

The dataset allows numerical and categorical sorting to be demonstrated on the same set of records.

6. Methodology

The project follows these steps:

1. Create a DataFrame containing student records.
2. Display the original dataset.
3. Sort marks in ascending order.
4. Sort marks in descending order.
5. Sort departments in ascending alphabetical order.
6. Sort departments in descending alphabetical order.
7. Sort by department ascending and marks descending.
8. Compare the sorting operations.
9. Export the original dataset to a CSV file.

7. Ascending Numerical Sorting

The program sorts the dataset by marks using:

`df.sort_values(by="marks", ascending=True)`

This arranges students from the lowest marks to the highest marks. In the sample dataset, Ishaan, with 68 marks, appears first, while Ananya, with 95 marks, appears last.

8. Descending Numerical Sorting

The program uses:

`df.sort_values(by="marks", ascending=False)`

This arranges students from the highest marks to the lowest marks. Ananya appears first, while Ishaan appears last.

9. Ascending and Descending Categorical Sorting

The dataset is sorted by department using ascending and descending alphabetical order.

Ascending sorting places the departments in the order Civil, Computer Science, Electronics, and Mechanical.

Descending sorting reverses this order to Mechanical, Electronics, Computer Science, and Civil.

10. Multiple-Column Sorting

The project demonstrates sorting by department and marks:

`df.sort_values(by=["department", "marks"], ascending=[True, False])`

The primary sort arranges departments alphabetically. The secondary sort places students with higher marks first within each department.

For example, the Computer Science records are ordered by marks from highest to lowest.

11. Results and Observations

The sorting operations produce the following results:

- Ascending marks sorting places the lowest mark first.
- Descending marks sorting places the highest mark first.
- Ascending department sorting arranges text alphabetically.
- Descending department sorting reverses alphabetical order.
- Multiple-column sorting applies a separate sorting direction to each selected column.
- Each operation retains all 12 original records.

Insert actual notebook output screenshots to document the results.

12. Applications

Sorting is useful for ranking students, organizing employee records, ordering sales by revenue, comparing product prices, arranging dates, and preparing data for further analysis.

Multiple-column sorting is especially useful when records need to be grouped by a category and then ordered by a numerical measure within each group.

13. Limitations

The project uses a small manually created dataset. It demonstrates basic sorting but does not cover advanced ranking, custom sorting rules, or large-scale data processing.

14. Future Enhancements

The project could be extended by sorting real datasets, allowing users to select columns interactively, applying custom sort rules, comparing group-wise results, and producing visualizations.

15. Conclusion

The project demonstrates the use of Pandas `sort_values()` to sort numerical and categorical data in ascending and descending order. It also shows how

multiple-column sorting supports more detailed organization of records.

These operations are useful in exploratory data analysis and in preparing datasets for further processing.

17. References

1. Pandas documentation: https://pandas.pydata.org/docs/
2. Pandas sorting reference: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.sort_values.html

18. Project Links

GitHub Repository: https://github.com/venky1318/Sort_Dataset